Fault Based Collision Attacks on AES

Johannes Blömer and Volker Krummel*

Faculty of Computer Science, Electrical Engineering and Mathematics
University of Paderborn, Germany
{bloemer, krummel}@uni-paderborn.de

Abstract. In this paper we present a new class of collision attacks that are based on inducing faults into the encryption process. We combine the classical fault attack of Biham and Shamir with the concept of collision attacks of Schramm et al. Unlike previous fault attacks by Blömer and Seifert our new attacks only need bit flips not bit resets. Furthermore, the new attacks do not need the faulty ciphertext to derive the secret key. We only need the weaker information whether a collision has occurred or not. This is an improvement over previous attacks presented for example by Dusart, Letourneux and Vivolo, Giraud, Chen and Yen or Piret and Quisquater. As it turns out the new attacks are very powerful even against sophisticated countermeasures like error detection and memory encryption.

1 Introduction

A smartcard is a general purpose computer embedded in a plastic cover of a credit card's size. The main building blocks of a smartcard are a CPU, a ROM that contains for example the operating system, an EEPROM containing among other things the secret key, and a RAM to store intermediate results of computations. To communicate with the outside world the smartcard has to be inserted into a so called smartcard reader that also provides the energy the smartcard needs for operating.

Smartcards are perfectly suited for storing private information such as cryptographic keys because the corresponding cryptographic operations such as encryption or digital signature are computed directly on the smartcard. Therefore the key never has to leave the smartcard and hence seems to be protected very well even in hostile environments. However, it is well known that physical instances of algorithms (in hardware or software) may leak information about the computation through so called side channels. Researchers identified several of those side channels and managed to use information obtained through side channels to determine secret keys of cryptographic applications. Kocher [13] was the first who presented an attack based on timing measurements that successfully computed the secret key of RSA in 1996. This result was improved by Dhem et al. [8]. Koeune and Quisquater [15] adapted timing attacks to the symmetric cipher AES. In 1999 Kocher, Jaffe and Jun [14] presented a successful side channel attack based on the power consumption of a smartcard.

* This work was partially supported by a grant from Intel Corporation, Portland.

L. Breveglieri et al. (Eds.): FDTC 2006, LNCS 4236, pp. 106–120, 2006.

In this paper we focus on so called fault attacks on the advanced encryption standard AES [7]. Boneh, DeMillo and Lipton [4] showed that faults induced into the encryption process of asymmetric ciphers can reveal the secret key. Biham and Shamir [1] combined fault attacks with the concept of differentials and mounted a differential fault attack (DFA) on DES. Skorobogatov and Anderson showed in [20] that fault attacks are realizable with sufficient precision in practice. See Blömer and Seifert [3] for an overview of the physics of inducing faults.

There are several fault attacks on AES reported in the literature. The first attacks were due to Blömer and Seifert [3] followed by improved attacks of Dusart, Letourneux and Vivolo [9], Giraud [10], Chen and Yen [6] and Piret and Quisquater [17]. All these publications demonstrate the power of fault attacks. However, these attacks either use the fault model of bit resets [3] in which case they do not need the faulty ciphertexts. Or the attacks only require the fault model of bit flips, in which case, however, the attacks need the faulty ciphertexts [9],[10],[6],[17]. The attacks presented in this paper use bit flips and, instead of faulty ciphertexts, the attacks only use so called collision information. This turns out to be a much weaker requirement than the requirement that an attacker gets complete faulty ciphertexts. To obtain our new attacks, we show how to combine fault attacks with so called collision attacks. In a collision attack the adversary tries to detect identical intermediate results during the encryption of different plaintexts, e.g., by using side channel information, and use this information to derive the secret key. Basically this idea was due to Dobbertin. Schramm et al. developed collision attacks against DES [19] and AES [18] and showed how to detect collisions using power traces.

We combine the concepts of fault and collision attacks by inducing faults to generate collisions. This approach allows to relax the requirement of getting faulty ciphertexts to the requirement of detecting collisions in the encryption process. First we explain the basic idea underlying our attacks by presenting an attack based on some rather strong assumptions. Then we present an attack utilizing the same basic ideas that successfully attacks a smartcard that is protected by a memory encryption mechanism. To the best of our knowledge, this is the first fault attack on smartcards protected by memory encryption.

To defend against side channel attacks the manufacturer invented several countermeasures. One type of countermeasure is intended to protect the card, e.g., shields, sensors or error detection. Another type is designed to render side channel attacks useless using techniques to obfuscate the side channel information, e.g. by random masking [16],[11],[2]. Yet another more efficient approach is to use a so called memory encryption mechanism (MEMO). Memory encryption mechanisms encrypt an intermediate result directly after it leaves the processor and decrypts data right before it enters the processor (see Figure 1). This guarantees that all data stored in the RAM is encrypted. The intention is that memory encryption makes it harder for an adversary to derive information about intermediate states of the encryption process by using side channels of the smartcard. In general, it is assumed that unlike the RAM the highly integrated processor is much to complicated to induce faults with some reasonable precision. Hence,

memory encryption is widely believed to be a useful countermeasure against side channel attacks.

Due to the limited computational power of smartcards the MEM has to be very fast. So the manufacturers of smartcards use some light encryption algorithms that are very fast but may not be secure against serious cryptanalysis. To increase the impact of the MEM the manufacturer like to keep their algorithms secret. However, many manufacturers do not analyze the impact of MEMs on security but simply present it as an improvement of security. The strategy is to implement as many good looking countermeasures as possible by not exceeding a certain cost threshold. Even a weak countermeasure should increase security.

Our attack that works even in the presence of a MEM shows that the security improvement of the MEM as generally used is rather limited. In particular, we present an attack on an AES implementation protected by MEM that determines the full AES-Key by inducing only 285 faults and detecting collisions.

The paper is organized as follows. In Section 2 we present our model for analyzing fault based collision attacks. In Section 3 we describe some fault based collision attacks and analyze their complexity. Unlike the classical fault attacks using bit flips in [9],[10],[6] and [17] obtaining faulty ciphertexts is not essential for our attacks. Therefore our attacks are applicable in scenarios where classical fault attacks do not work. On the other hand, our new attacks need more faults than the classical fault attacks. We explain the basic idea in our first attack. This attack is our basic attack and is based on rather strong assumptions. However, in the sequel we show how to strengthen it and how to adapt it to several other scenarios. The second attack we present is is our strongest attack. This attack shows how to successfully attack a smartcard that is protected by a MEM. To the best of our knowledge this is the first successful attack against a smartcard protected by a MEM.

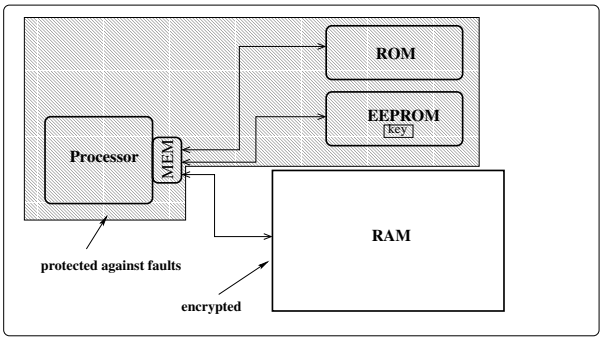

Fig. 1. Model of an enhanced smartcard with memory encryption mechanism (MEM)

2 Model

In our scenario we have a smartcard with an implementation of AES and a secret AES key K stored on it. We simplify the real world by assuming that only the RAM may leak some information and all other parts are well protected. An adversary

$\mathcal{A}$ is able to input chosen plaintexts and induce faults in terms of bit flips into the RAM in order to derive some information about the secret key K. To be more precise, $\mathcal{A}$ can flip a single bit of some specified byte in the memory and derive so called collision information about an internal state of the encryption process.

We regard the AES encryption as a bijective function AES_K that maps a plaintext p on a ciphertext depending on the secret key $K = (k_0, \ldots, k_{15})$ [1]. To model faults mathematically we extend that function with a second variable b that specifies a bit position during the computation of AES_K.

The set of all realizable functions via AES is extended by flipping bit b during the computation of AES_K. However, the extended function $\mathrm{FAES}_K(p, b)$ is not bijective. So there exist collisions such that two intermediate states of computations of $\mathrm{FAES}_K(p, b)$ and $\mathrm{FAES}_K(p', b')$ with different inputs $(p, b) \neq (p', b')$ are equal. An attacker wants to detect those collisions and then use them to derive the secret key K.

We state three assumptions. First, $\mathcal{A}$ is able to feed chosen[2] plaintexts into the encryption algorithm. Second, $\mathcal{A}$ is able to induce faults in terms of bit flips into a specific bit of the RAM. Our third assumption is that $\mathcal{A}$ is able to derive some information about an intermediate state of the encryption process. However, we do not assume that this information lets $\mathcal{A}$ determine (parts of) the secret key directly. Nevertheless it enables $\mathcal{A}$ to detect if a collision occurred or not. We call any kind of information that lets $\mathcal{A}$ detect collisions *collision information* of some intermediate state of $\mathrm{FAES}_K(p, b)$. Later we will show examples how to derive collision information.

We model collision information as the evaluation of an injective function f_K that depends on the concrete implementation of AES_K and the secret key K. It gets as input a plaintext p, the time t when a bit flip occurs, the byte position x and the bit position b inside that byte. The output is some information $f_K(p, t, x, b)$ about an intermediate state of the encryption. Certainly it is also possible to derive the collision information without inducing a fault.

Depending on the purpose of the smartcard f_K can have different realizations. Given the ciphertexts the detection of collisions is easy because the equality of ciphertexts implies equality of intermediate states. However, in many cases the output of an encryption is not available to the attacker. For example, if the smartcard computes a CBC-MAC or a hash value using AES as a building block $f_K(p, b)$ can simply be the MAC / hash value. Remember that the MAC is the final result of a number of interlinked AES encryptions and not the result of a single AES encryption. The final ciphertext could also be used as collision information if the smartcard computes multiple encryption with different encryption algorithms. Finally, if the smartcard computes a single encryption but does not output faulty ciphertexts, f_K could be the measurement of some side channel information, e.g., power consumption profile, that allows to detect collisions.

[1] For simplicity we only consider AES-128. However, all the attacks in this paper can easily be adapted to AES with larger key sizes.

[2] The attacks presented in this paper can also be transformed to known plaintext attacks.

To analyze the cost of an attack we simply count the number of faults we have to induce. The evaluation of f_K without inducing a fault is for free. We also neglect the complexity of additional computations that can be performed offline since in our cases they are obviously easy.

3 New Fault Attacks

3.1 Notation

For simplicity and clarity we define $p_i^{(r),(o)}$ to be the ith byte of the encryption state of plaintext p after the operation o of round r. The operation o is one of the following:

B SubBytes
R ShiftRows
C MixColumns
K AddRoundkey

For example, $p_5^{(3),(R)}$ is the 5th byte of the encryption state of plaintext p after the ShiftRows operation of round 3. The ith byte of the round key of round r is called $k_i^{(r)}$. We denote the transformation of SubBytes applied on a single byte x of the state simply as the application of the *sbox* on x and write it as $\mathbf{S}[x]$. To simplify notation we define $\Delta(p_i, q_i) = p_i + q_i$ to be the difference of two plaintext bytes p_i and q_i. Then $\Delta_{in}(p_i, q_i) = (p_i + k_i^{(0)}) + (q_i + k_i^{(0)}) = p_i + q_i$ is the input difference of (p_i, q_i) before the first application of the sbox and $\Delta_{out}(p_i, q_i) = (\mathbf{S}[p_i + k_i^{(0)}] + \mathbf{S}[p_i + k_i^{(0)}])$ is the output difference of (p_i, q_i) after the first application of the sbox. To simplify notation further we denote the collision information of encrypting plaintext p and inducing a bit flip into bit e of byte 0 of the state after the application of SubBytes in round 1 by $f_K(p_0^{(1),(B)}, e)$. We denote the evaluation of f_K without inducing a fault in the encryption process by $f_K(p_0^{(1),(B)}, -)$. From the context it will always be clear which plaintext is meant.

3.2 Scenarios

Below we describe some attacks that are based on the detection of collisions. For simplicity, we only show how to compute byte 0 of the secret key. Similar approaches can be used to compute the other key bytes.

We describe how to mount and analyze each attack in different scenarios. Each scenario is characterized by abilities of the adversary and/or the environment. The first characteristic defines the precision of the fault induction. We look at the two cases that the adversary is able to flip a specific bit of an intermediate state and that each possible bit flip occurs with probability 1/8.

The second characteristic specifies whether the smart card is protected by a MEM (memory encryption mechanism) or not. The MEM encrypts every intermediate result that leaves the processor and decrypts a value right before it enters the processor (see Figure 1). Since a smart card has only restricted

computational power and memory most manufacturers choose a byte oriented encryption function with a fixed key that is used for encryption and decryption. In our approach we simply model the memory encryption as an unknown but fixed function $h : \{0,1\}^8 \to \{0,1\}^8$. That means that we do not rely on a weakness in the memory encryption itself. In particular, we do not assume to have any information of how bit flips effect further processing of that byte.

The last characteristic defines whether collision information remains valid for a long period of time or not. If collision information does not remain valid there is no reason for $\mathcal{A}$ to store collision information since he cannot use it later in the attack. $\mathcal{A}$ is only able to compare collision information of two recently taken measurements and store the result. This effect could be caused by environments that are frequently changed such that collision information taken at different times is hardly comparable, e.g., due to some countermeasure that induces noise into the collision information. If, however, collision information remains valid over the time segment used for the attack it maybe useful for $\mathcal{A}$ to store this information in a preprocessing step to have it available once and for all. As we will see later stored information is useful as it helps to reduce the number of faults.

3.3 First Attack

First, we describe the scenario in which the attack takes place. We assume that $\mathcal{A}$ can flip a specific bit e of the intermediate state $p^{(1),(B)}$. We also assume that collision information remains valid over the time span of the attack. Finally, we assume that the smartcard is not protected by a MEM.

In a preprocessing step the adversary computes an array T_e of length 256. In position $T_e[y], y \in \{0,\ldots,255\}$ the array stores the following information:

$$T_e[y] := \big\{\{s,t\} \,\big|\, s+t = y, \mathbf{S}[s] + \mathbf{S}[t] = 2^e\big\},$$

i.e., $T_e[y]$ stores all (unordered) pairs of bytes with $\Delta_{in}(s,t) = y$ and $\Delta_{out}(s,t) = \mathbf{S}[s] + \mathbf{S}[t] = 2^e$. Furthermore, by $C_e[y]$ denote the union of sets in $T_e[y]$. The sets $C_e[y]$ are pairwise disjoint. As it turns out, for every $e \in \{0,1,\ldots,7\}$ we have that 129 sets $C_e[y]$ are empty, 126 sets $C_e[y]$ contain exactly two elements, and one set $C_e[y]$ contains exactly four elements.

Next, $\mathcal{A}$ collects a set T of collision information $f_K(p_0^{(1)(B)}, -)$ for all 256 different values of p_0 and arbitrary but fixed $p_1,\ldots,p_{15}$. Then $\mathcal{A}$ chooses an arbitrary value q_0 and encrypts the corresponding plaintext flipping an arbitrary bit e of $q_0^{(1),(B)}$. If f_K has the property that $f_K(p_0^{(1),(B)}, -) = f_K(q_0^{(1),(B)}, e)$ $\mathcal{A}$ is able to find the corresponding plaintext p_0 satisfying $\mathbf{S}[p_0 + k_0] = \mathbf{S}[q_0 + k_0] + 2^e$ by comparing the collision information with the elements of T. Given the pair p_0, q_0 the adversary knows the difference $p_0 + k_0 + q_0 + k_0 = p_0 + q_0$. Using array T_e the adversary $\mathcal{A}$ now concludes $\{p_0 + k_0, q_0 + k_0\} \in T_e[p_0 + q_0]$. Hence, $\mathcal{A}$ knows that the correct key byte k_0 satisfies

$$k_0 \in \big\{p_0 + s \,\big|\, s \in C_e[p_0 + q_0]\big\}. \tag{1}$$

As mentioned above, $\left|C_e[y]\right| \leq 4$ for all y, and $\left|C_e[y]\right| = 2$ for all but one y. Hence, at this point $\mathcal{A}$ has reduced the number of possible values for key byte k_0 to at most 4.

Next, the adversary repeats the experiment described above with some value q_0', such that $q_0' + s \notin C_e[p_0 + q_0]$ for all $s \in \{p_0 + \bar{s} \,|\, \bar{s} \in C_e[p_0 + q_0]\}$. Using the collision information in set T, the adversary determines p_0' such that $\mathbf{S}[p_0' + k_0] = \mathbf{S}[q_0' + k_0] + 2^e$. As before $\mathcal{A}$ concludes that the key byte k_0 satisfies

$$k_0 \in \left\{p_0' + s \,\middle|\, s \in C_e[p_0' + q_0']\right\}. \tag{2}$$

By choice of q_0', the adversary $\mathcal{A}$ is guaranteed that $p_0 + q_0 \neq p_0' + q_0'$. By elementary arithmetic it follows that if $\left|C_e[p_0' + q_0']\right| = \left|C_e[p_0 + q_0]\right| = 2$, then (1) and (2) uniquely determine the key byte k_0. As it turns out, the same is true if one of the sets has size four. However, to verify this, one has to perform a tedious case analysis based on the exact structure of the arrays T_e. We omit this in this extended abstract.

Cost Analysis To determine a single AES key byte $\mathcal{A}$ has to induce two faults. Thus 32 faults are enough to determine the full 128-bit AES key.

3.4 Second Attack

The scenario for this attack is as follows. We assume that $\mathcal{A}$ can flip a specific bit e of the intermediate state $p^{(0),(K)}$. We also assume that collision information remains valid over the time span of the attack. Finally, we assume that the smartcard is protected by a MEM modelled as a function $h : \{0,1\}^8 \to \{0,1\}^8$. This implies that after a flip of bit e the encryption continues using the value $h^{-1}(h(p_i + k_i) + 2^e)$ instead of $p_i + k_i$. Therefore, we assume that we have no information about the impact of bit flips on the encryption process.

The attack is divided into two steps. In the first step $\mathcal{A}$ collects the necessary information to compute a function g_0 that is equal to h up to some constant coefficient. To do so $\mathcal{A}$ selects a set S of 256 plaintexts p that take on all different values in byte p_0 and that are equal in each other byte. $\mathcal{A}$ uses the smartcard to derive the collision information for each of these plaintexts by evaluating $f_K(h(p_0^{(0)(K)}), -)$ and stores it in the table T. Then $\mathcal{A}$ encrypts plaintexts p of the set S and induces a bit fault into bit $0 \leq e \leq 7$ of $h(p_0^{(0),(K)})$ and compares the collision information $f_K(h(p_0^{(0),(K)}), e)$ with the entries of table T to find the corresponding plaintext p_0'. So $\mathcal{A}$ knows the difference

$$h(p_0 + k_0) + h(p_0' + k_0) = 2^e$$

and stores the triple (p_0, p_0', e) in a difference table DT. This step is repeated for different plaintexts p and for different faulty bit positions until $\mathcal{A}$ has enough information to compute the differences

$$h(p_0 + k_0) + h(p_0' + k_0)$$

of one byte p_0 with all other bytes p_0'. The details are given in the following lemma.

Lemma 1. *Let $m : \{0,1\}^q \to \{0,1\}^q$ be an unknown function defined over $\mathbb{F}_{2^q}$. There exists a set D of $2^q - 1$ pairs $(u, v) \in \mathbb{F}_{2^q} \times \mathbb{F}_{2^q}$ with the following property: If for all $(u, v) \in D$ we have that $m(u) + m(v) = 2^e$ for some known $e \in \{0, \ldots, q-1\}$, then one can determine a function g such that $g + c = m$ for some constant $c \in \mathbb{F}_{2^q}$.*

Proof. Given some set $D \subseteq \mathbb{F}_{2^q} \times \mathbb{F}_{2^q}$ we construct a graph G whose set of vertices is $\mathbb{F}_{2^q}$ as follows. We connect two vertices u, v with an edge of weight e if $(u, v) \in D$.

If in G there exists a path between two vertices x, y then the difference $m(x)+m(y)$ is determined by the differences of pairs in D. Furthermore, if the graph G is connected we can compute the difference $m(x)+m(y)$ for all $(x, y) \in \mathbb{F}_{2^q} \times \mathbb{F}_{2^q}$. In particular, we can determine all differences of the form $m(u) + m(u_0)$ for an arbitrary but fixed input u_0. Then using Lagrange interpolation we can now compute the function $g(u) = m(u) + m(u_0)$. Setting $c := m(u_0)$ proves the lemma.

Next we describe a set D of pairs (u, v) with known differences $m(u)+m(v) = 2^e$, such that the graph G as defined above is in fact connected. First we fix an arbitrary $e_1 \in \{0, \ldots, q-1\}$. Then there exists a set D_1 of 2^{q-1} distinct pairs $(u, v) \in \mathbb{F}_{2^q} \times \mathbb{F}_{2^q}$ such that $m(u) + m(v) = 2^{e_1}$. All pairs in D_1 will be elements of D. If we consider the graph whose edges are defined by pairs in D_1 we get a graph G_1 on the vertex set $\mathbb{F}_{2^q}$ that consists of 2^{q-1} connected components each consisting of exactly 2 vertices.

Next we choose $e_2 \neq e_1$. Then there exists a set D_2 of 2^{q-2} pairs of vertices (u, v) with $m(u) + m(v) = 2^{e_2}$ such that each pair in D_2 connects different connected components of G_1. We call the resulting graph G_2. The set D will also contain all elements from D_2.

Continuing in this way with all possible $e_i \in \{0, \ldots, q-1\}$ we get sets of pairs $D_1, D_2 \ldots, D_q$ and graphs $G_1, G_2, \ldots, G_q$ such that G_i has 2^{q-i} connected components. In particular, G_q is connected. Moreover, the edges of G_q are given by the pairs in $D := \bigcup_{i=1}^{q} D_i$. The size of D is $2^q - 1$. This proves the lemma.

We want to apply Lemma 1 to the function $h(x + k_0)$. It is easy to see that $\mathcal{A}$ can compute exactly the set of differences D described in the proof of Lemma 1 since he is able to flip a specific bit. Hence, knowing D the adversary $\mathcal{A}$ can compute a function $g_0 : \{0,1\}^8 \to \{0,1\}^8$ such that for all $x \in \mathbb{F}_{256}$ the difference $g_0(x)+h(x+k_0)$ is some constant $c_0 \in \mathbb{F}_{256}$. Since $\mathcal{A}$ does not know the constant c_0 he does not get any information about the key byte k_0 at this point.

$\mathcal{A}$ continues by computing for all other byte positions i a function $g_1, \ldots, g_{15}$ such that for all $x \in \mathbb{F}_{256}$ the function $g_i : \{0,1\}^8 \to \{0,1\}^8$ has the property that $g_i(x) + h(x + k_i) = c_i$ for some unknown constant $c_i \in \mathbb{F}_{256}$. Each of the g_i's does not reveal any information about the involved key byte k_i because the constant c_i can take on all possible values and is unknown to $\mathcal{A}$.

To derive information about the key $\mathcal{A}$ proceeds as follows. He guesses two candidates $\widehat{k_0}, \widehat{k_i}$ for the keybytes k_0, k_i, respectively. To test this hypothesis on the key $\mathcal{A}$ selects several bytes x uniformly at random and computes

$$g_0(x + \widehat{k}_0) = h(x + \widehat{k}_0 + k_0) + c_0$$

and

$$g_i(x + \widehat{k}_i) = h(x + \widehat{k}_i + k_i) + c_i.$$

Depending on the hypothesis $(\widehat{k}_0, \widehat{k}_i)$ the difference $t_{0,i} := g_0(x + \widehat{k}_0) + g_i(x + \widehat{k}_i)$ computes to

$$h(x) + c_0 + h(x) + c_i = c_0 + c_i \quad \text{,if } \widehat{k}_0 + k_0 = \widehat{k}_i + k_i \tag{3}$$

$$h(x + \widehat{k}_0 + k_0) + c_0 + h(x + \widehat{k}_i + k_i) + c_i \quad \text{,if } \widehat{k}_0 \neq k_0 \text{ and } \widehat{k}_i \neq k_i \tag{4}$$

$$h(x) + c_0 + h(x + \widehat{k}_i + k_i) + c_i \quad \text{,if } \widehat{k}_0 = k_0 \text{ and } \widehat{k}_i \neq k_i \tag{5}$$

$$h(x + \widehat{k}_0 + k_0) + c_0 + h(x) + c_i \quad \text{,if } \widehat{k}_0 \neq k_0 \text{ and } \widehat{k}_i = k_i \tag{6}$$

Now we assume that the function h has the following property. There do not exist constants $a, c \in \mathbb{F}_{256}$ such that $h(x) + a = h(x + c)$ for all x. Note that this assumption does not restrict the choice of h for two reasons. First, a function used for memory encryption that does not have this property contains too much structure and is probably easier to attack. Secondly, most functions have this property. In fact, a random function has the property with probability at least $1 - 2^{-127}$.

This assumption implies that unlike in case (3) in cases (4),(5),(6) the difference $t_{0,i}$ is not constant. Moreover, if the guess $\widehat{k}_0, \widehat{k}_i$ was correct that is $\widehat{k}_0 = k_0$ and $\widehat{k}_i = k_i$ then $\mathcal{A}$ will always be in case (3). Now $\mathcal{A}$ can easily test the hypothesis $(\widehat{k}_0, \widehat{k}_1)$ by computing $t_{0,i}$ for several bytes x. If $t_{0,i}$ varies for several different x then $\mathcal{A}$ knows that he is not in case (3). It follows that the pair $(\widehat{k}_0, \widehat{k}_1)$ cannot be correct. On the other hand if $t_{0,i}$ remains constant $\mathcal{A}$ concludes to be in case (3) and keeps the pair $(\widehat{k}_0, \widehat{k}_1)$ as a potentially correct candidate.

This implies that for every possible key byte $\widehat{k}_0$ the adversary $\mathcal{A}$ obtains a single candidate $\widehat{k}_i$ for $1 \leq i \leq 15$ that fulfills condition (3). Guessing $\widehat{k}_0$ the adversary $\mathcal{A}$ can compute a vector $(\widehat{k}_1, \ldots, \widehat{k}_{15})$ composed of unique candidates $\widehat{k}_i$ that only depend on $\widehat{k}_0$. To uniquely determine the correct key $\mathcal{A}$ simply mounts an exhaustive search attack on the 256 possible values of $\widehat{k}_0$.

Cost Analysis. $\mathcal{A}$ has to induce 255 faults to compute a function g_i according to Lemma 1. To test a hypothesis of the key $\mathcal{A}$ does not need to induce faults. So the overall number of faults is $16 \cdot 255 = 4080$.

Improvement. The previous attack can be improved with respect to the number of induced faults as shown below. In the first step $\mathcal{A}$ computes the function g_0 such that $g_0(x) = h(x + k_0) + c_0$, where $c_0 \in \mathbb{F}_{256}$ is unknown, as above. To determine the other functions $g_1, \ldots, g_{15}$ $\mathcal{A}$ uses the fact that each g_i is related to g_0 by the following equation

$$g_i(x) = h(x + k_i) + c_i = g_0(x + \underbrace{k_i + k_0}_{s_i}) + c_i + c_0.$$

So knowing g_0 (determined as above) $\mathcal{A}$ computes a list of all 256 functions $g_{0,s} := g_0(x+s)$, $s \in \mathbb{F}_{256}$. To determine which of these functions equals g_i the adversary $\mathcal{A}$ chooses arbitrary p_i, q_i and evaluates $f_K(h(p_i^{(0),(K)}), -)$ and $f_K(h(q_i^{(0),(K)}), e)$ at byte position i. Using this information $\mathcal{A}$ computes some differences $g_i(p_i) + g_i(q_i)$ as described in the computation of g_0 above.

To determine the correct function $g_i = g_{0,s_i}$ $\mathcal{A}$ simply checks which of the function $g_{0,s}$ fulfills these differences simultaneously until only one function remains. See below for the required number of experiments. Then $\mathcal{A}$ knows the sum $s_i = k_0 + k_i$ of two AES key bytes. $\mathcal{A}$ repeats this procedure for all other byte positions $0 \leq i \leq 15$. As before guessing $\widehat{k_0}$ the adversary $\mathcal{A}$ can determine a unique candidate $\widehat{k_i}$. That means that $\mathcal{A}$ has a vector $(\widehat{k_1}, \ldots, \widehat{k_{15}})$ with fixed candidates $\widehat{k_i}$ for each of the 256 candidates $\widehat{k_0}$. Like in the original version of this attack this reduces the set of possible AES keys to only 256 candidates. An exhaustive search reveals the full AES key.

Cost Analysis. To compute g_0 the adversary $\mathcal{A}$ has to induce 255 faults like in the original version. To determine further g_i's $\mathcal{A}$ has to collect a set of differences $g_i(p)+g_i(q)$ that is fullfilled by only one of the 256 functions $g_{0,s}$ simultaneously. Notice that if the function $g_{0,s}$ fulfills a difference, i.e., $g_0(p+s) + g_0(q+s) = g_i(p)+g_i(q)$ then because of symmetry the function $g_{0,s'}$ given by $s' := p+q+s$ also fulfills this difference since

$$g_0(p + (p+q+s)) + g_0(q + (p+q+s) = g_0(q+s) + g_0(p+s) = g_i(q) + g_i(p).$$

Assuming that the 256 functions $g_{0,s}$ behave like random permutations (except for the symmetry) we expect that $\mathcal{A}$ needs 2 differences to uniquely identify the correct one with high probability. We tested this assumption by various experiments and in our experiments it proved to be correct. Hence, we expect that $\mathcal{A}$ needs $255 + 15 \cdot 2 = 285$ faults to determine the full AES key.

As mentioned before we do not consider the complexity of the offline calculations like Lagrange interpolation etc. since all these calculations are easy to perform.

3.5 Third Attack

First, we describe the scenario in which the attack takes place. We assume that $\mathcal{A}$ can flip a specific bit e of the intermediate state $p^{(1),(B)}$. We do not assume that collision information remains valid over the time span of the attack. Hence, $\mathcal{A}$ is only able to compare collision information of two recently obtained measurements. Finally, we assume that the smartcard is not protected by a MEM. Because it is always clear from the context we simplify notation by identifying elements of $\mathbb{F}_{256}$ with their canonical representation as elements of the set $\{0, \ldots, 255\}$.

As a basis for his attack $\mathcal{A}$ fixes some input difference Δ_{in} and output difference Δ_{out} of the application of the sbox in round 1. To be able to detect collisions with a single bit flip we restrict Δ_{out} to be a power of 2.

The analysis of the sbox shows that there are a lot of suitable values for Δ_{in} and Δ_{out} (see technical analysis in the full version of the paper). E.g. $\mathcal{A}$ chooses $\Delta_{in} = 10$ and $\Delta_{out} = 4$. Only the two pairs

$$Z_1 := (p_0 + k_0 = 0, q_0 + k_0 = 10)$$

and

$$Z_2 := (p_0 + k_0 = 244, q_0 + k_0 = 254)$$

together with their commuted counterparts fulfill the chosen requirements. A fault that is induced into bit 2 of $q_0^{(1),(B)}$ after the application of the sbox results in a collision for one of these pairs. In order to detect such a collision the collision information f_K should have the property that

$$f_K(p_0^{(1),(B)}, -) = f_K(q_0^{(1),(B)}, 2).$$

If $\mathcal{A}$ finds such a collision he can conclude that the key byte k_0 is an element of the set

$$\mathcal{K} = \{p_0 + 0, p_0 + 10, p_0 + 244, p_0 + 254\}$$

More precisely, the attack using f_K with the property defined above works as follows. First, the adversary $\mathcal{A}$ generates all 128 pairs of plaintexts (p, q) (without symmetry) that have difference 10 in byte 0 ($p_0 = q_0 + 10$) and are equal in the other bytes, i.e.,

$$\Delta(p_i, q_i) = \begin{cases} 10, \text{ if i=0} \\ 0, \text{ otherwise} \end{cases}$$

$\mathcal{A}$ knows that exactly two of these pairs have output difference 4 in byte 0. The input difference of the sbox is the same as the difference of p_0 and q_0 since AddRoundKey does not change it. $\mathcal{A}$ checks all 128 pairs (p, q) until

$$f_K(p_0^{(1),(B)}, -) = f_K(q_0^{(1),(B)}, 2).$$

Taking the symmetry into account it follows that either $p_0+k_0 = 0$, $p_0+k_0 = 10$, $p_0 + k_0 = 244$ or $p_0 + k_0 = 254$. So there are only 4 candidates for k_0 left. $\mathcal{A}$ can repeat this attack for all byte positions of the state. This leaves $2^{2 \cdot 16} = 2^{32}$ possible keys. To determine the complete 128-bit AES key $\mathcal{A}$ mounts an exhaustive search attack.

Cost Analysis In the first step $\mathcal{A}$ examines 128 pairs of plaintexts with difference 10. Two of these pairs result in a collision so the expected number of faults $\mathcal{A}$ has to induce is $(2/128)^{-1} = 64$. To compute a 128 bit AES key $\mathcal{A}$ expects to induce $16 * 64 = 1024$ faults and a brute force attack of size 2^{32}.

Alternative To determine the correct candidate of the key byte $\mathcal{A}$ could also repeat the same procedure as above with another difference. We assume that f_K lets $\mathcal{A}$ detect collisions when flipping bit 3, i.e.

$$f_K(p'^{(1),(B)}_0, -) = f_K(q'^{(1),(B)}_0, 3).$$

If we look at all pairs at all pairs (p', q') such that

$$\Delta(p'_i, q'_i) = \begin{cases} 5, \text{ if i=0} \\ 0, \text{ otherwise} \end{cases}$$

an analysis of the sbox shows that $Z_3 := (p'_0 + k_0 = 0, q'_0 + k_0 = 5)$ and $Z_4 := (p'_0 + k_0 = 122, q'_0 + k_0 = 127)$ are the only pairs with $\Delta_{in} = 5$ and $\Delta_{out} = 8$. Detecting one of these pairs using f_K yields again a set of 4 candidates for k_0.

Next, $\mathcal{A}$ computes the difference of plaintexts p_0 and p'_0. The difference must be one of the differences listed in Table 1. Since all possible differences are distinct $\mathcal{A}$ can determine $p_0 + k_0$ and hence k_0.

Table 1. All possible differences of p_0,p'_0

	$p_0 + k_0$			
$p'_0 + k_0$	0	10	244	254
0	0	10	244	254
5	5	15	241	251
122	122	112	142	132
127	127	117	139	129

Cost Analysis. Following the cost analysis as above this method determines the correct candidate of each key byte with 1024 faults as in the previous method plus additional 1024 faults.

3.6 Fourth Attack

First, we describe the scenario in which the attack takes place. We assume that $\mathcal{A}$ can flip a bit of a specific byte of the intermediate state $p^{(1),(B)}$. However, he has no control over the bit position. Instead, we assume that all of the 8 possible bit flips occur with the same probability 1/8. We also assume that collision information remains valid over the time span of the attack. Finally, we assume that the smartcard is not protected by a MEM.

The attack works as follows. In a first step $\mathcal{A}$ selects a set S of 256 plaintexts p that take on all different values in byte p_0 and are equal in each other byte. $\mathcal{A}$ collects the collision information $f_K(p_0^{(1),(B)}, -)$ for all elements of S. Then he chooses an arbitrary q_0 and encrypts the corresponding plaintext inducing a fault into bit e of $q_0^{(1),(B)}$. By comparing the collision information $f_K(q_0^{(1),(B)}, e)$ with the collision information collected in the first step $\mathcal{A}$ can determine the corresponding plaintext p_0 such that $\mathbf{S}[p_0 + k_0] = \mathbf{S}[q_0 + k_0] + 2^e$. Note that e is unknown to $\mathcal{A}$ since he does not have any influence on the bit position. $\mathcal{A}$ can test all candidates $\widehat{k}_0$ of k_0 by simply checking if $\mathbf{S}[p_0 + \widehat{k}_0] + \mathbf{S}[q_0 + \widehat{k}_0]$ is a power of 2. If this condition is true $\mathcal{A}$ stores $\widehat{k}_0$ as a possible key value and discard it otherwise. Analysis of the AES sbox shows that after checking all candidates a set of at most 16 candidates will remain. $\mathcal{A}$ repeats this procedure with different q_0 until only one candidate is left. Using a refined method similar to the attack in Section 3.3 using several different q_0 we can determine the correct key.

3.7 Fifth Attack

First, we describe the scenario in which the attack takes place. We assume that $\mathcal{A}$ can flip a bit of a specific byte of the intermediate state $p^{(1),(B)}$. However, he

has no control over the bit position. Instead, we assume that all of the 8 possible bit flips occur with the same probability 1/8. We do not assume that collision information remains valid over the time span of the attack. Hence, $\mathcal{A}$ is only able to compare collision information of two recently obtained measurements. Finally, we assume that the smartcard is not protected by a MEM.

$\mathcal{A}$ chooses Δ_{in} of the sbox in round 1 in such a way that the number of pairs that have difference Δ_{in} and output difference with Hamming weight 1 is maximal. This choice reduces the number of faults $\mathcal{A}$ has to induce as we will see later. Analysis of the sbox shows that $\Delta_{in} = 216$ is the best choice since 8 is the maximum number of pairs that fulfill the requirements.

A single bit flip induced into $q_0^{(1)(B)}$ may produce a collision if and only if $p_0 + k_0$ is one of the following values:

$$0, 2, 8, 28, 29, 41, 111, 117, 173, 183, 196, 197, 208, 216, 218, 241.$$

To detect the collision f_K should have the property that

$$f_K(p_0^{(1)(B)}, -) = f_K(q_0^{(1)(B)}, b) \tag{7}$$

A collision implies that k_0 is an element of the set of 16 candidates

$$\begin{aligned}\mathcal{K} = \{\ & p_0, p_0+2, p_0+8, p_0+28, p_0+29, p_0+41, p_0+111, p_0+117, p_0+173,\\ & p_0+183, p_0+196, p_0+197, p_0+208, p_0+216, p_0+218, p_0+241\}.\end{aligned}$$

To determine p_0 the adversary $\mathcal{A}$ first builds a list of all 128 pairs (p_0, q_0) of plaintexts with difference 216 in byte 0 and difference 0 in all other bytes. Then $\mathcal{A}$ selects an arbitrary q_0, derives $f_K(q_0^{(1)(B)}, b)$ of the corresponding plaintext and compares it with the collision information $f_K(p_0^{(1)(B)}, -)$ of the corresponding plaintext of p_0. $\mathcal{A}$ repeats this procedure until he detects a collision. At his point $\mathcal{A}$ knows that k_0 is an element of the set $\mathcal{K}$.

To identify the correct candidate $\mathcal{A}$ could start an exhaustive search or repeat the procedure with a different combination of input and output differences. For example $\mathcal{A}$ chooses input difference 4 and output difference 32. Since $(88, 92)$ is the only such pair $\mathcal{A}$ can use f_K as a special case of (7) having the property

$$f_K(p_0^{(1)(B)}, -) = f_K(q_0^{(1)(B)}, 5)$$

to test each candidate $\widehat{k}_0 \in \mathcal{K}$ of k_0.

To check whether a candidate $\widehat{k}_0 \in \mathcal{K}$ is equal to k_0, $\mathcal{A}$ derives the collision information $f_K(p_0^{(1)(B)}, -)$ and $f_K(q_0^{(1)(B)}, b)$ for $p_0 = \widehat{k}_0 + 92$ and $q_0 = \widehat{k}_0 + 88$. Since $(92, 88)$ is the only pair with input difference 4 and Hamming weight of the output difference 1 $\mathcal{A}$ can check his hypothesis $\widehat{k}_0$. More precisely if $\widehat{k}_0 \neq k_0$ the Hamming weight of the output difference will always be greater than 1 except for the case that $p_0^{(0)(K)} = 88$ and $q_0^{(0)(K)} = 92$. But this case implies that $\widehat{k}_0 + 4 = k_0$ which is impossible since every difference of two of the sixteen candidates is different from 4. So a wrong hypothesis cannot create a collision. On the other hand if $\widehat{k}_0 = k_0$ then $p + k_0 = 92 + \widehat{k}_0 + k_0 = 92$ and $q + k_0 = 88 + \widehat{k}_0 + k_0 = 88$ is the demanded pair and $\mathcal{A}$ will detect a collision using f_K.

Cost Analysis. The success probability of finding one of the 8 pairs in part one of the attack choosing p_0 uniformly at random is $\frac{8}{128} \cdot \frac{1}{8} = \frac{1}{128}$. Hence 128 is the expected number of faults $\mathcal{A}$ has to induce.

The success probability in the second step is $(1/8) \cdot (1/16) = 1/128$. So we expect that $\mathcal{A}$ needs additional 128 faults. Hence the total number of faults to determine a key byte is $2 \cdot 128 = 256$.

To compute a complete 128 bit AES key we expect that $\mathcal{A}$ needs $16 \cdot 256 = 4096$ faults.

4 Concluding Remarks

In this paper we introduced the concept of fault based collision attacks that is a combination of collision attacks with fault attacks. We also showed how to mount fault based collision attacks on AES. Thereby we considered so called memory encryption mechanisms (MEM), a widely used countermeasure to protect against side channel attacks. We showed that using MEM in a straightforward manner does not increase security as much as one would expect. E.g., we presented a fault based collision attack that breaks an implementation protected by a MEM by inducing only about 285 faults.

To thwart our attack one has to be more careful. For example using different MEM functions for different bytes of a state obviously renders our attack useless. An alternative and more general approach is to use a general randomization strategy such as [2] based on [5].

References

1. Eli Biham and Adi Shamir. Differential fault analysis of secret key cryptosystems. In Burton S. Kaliski Jr., editor, *CRYPTO*, volume 1294 of *Lecture Notes in Computer Science*, pages 513–525. Springer, 1997.
2. Johannes Blömer, Jorge Guajardo, and Volker Krummel. Provably secure masking of AES. In H. Handschuh and M. Anwar Hasan, editors, *Proceedings Selected Areas in Cryptography (SAC), Lecture Notes in Computer Science Volume 3357*, pages 69–83. Springer-Verlag, 2004.
3. Johannes Blömer and Jean-Pierre Seifert. Fault based cryptanalysis of the advanced encryption standard (AES). In *Financial Cryptography'03, Lecture Notes in Computer Science Volume 2742*, pages 162–181. Springer-Verlag, 2003.
4. Dan Boneh, Richard A. DeMillo, and Richard J. Lipton. On the importance of checking cryptographic protocols for faults (extended abstract). In *EUROCRYPT*, pages 37–51, 1997.
5. Suresh Chari, Charanjit S. Jutla, Josyula R. Rao, and Pankaj Rohatgi. Towards sound approaches to counteract power-analysis attacks. In Wiener [21], pages 398–412.
6. Chien-Ning Chen and Sung-Ming Yen. Differential fault analysis on AES key schedule and some countermeasures. In Reihaneh Safavi-Naini and Jennifer Seberry, editors, *ACISP*, volume 2727 of *Lecture Notes in Computer Science*, pages 118–129. Springer, 2003.

7. Joan Daemen and Vincent Rijmen. *The Design of Rijndael*. Information Security and Cryptography. Springer Verlag, 2002.
8. Jean-François Dhem, François Koeune, Philippe-Alexandre Leroux, Patrick Mestré, Jean-Jacques Quisquater, and Jean-Louis Willems. A practical implementation of the timing attack. In Jean-Jacques Quisquater and Bruce Schneier, editors, *CARDIS*, volume 1820 of *Lecture Notes in Computer Science*, pages 167–182. Springer, 1998.
9. Pierre Dusart, Gilles Letourneux, and Olivier Vivolo. Differential fault analysis on A.E.S. In Jianying Zhou, Moti Yung, and Yongfei Han, editors, *ACNS*, volume 2846 of *Lecture Notes in Computer Science*, pages 293–306. Springer, 2003.
10. Christophe Giraud. DFA on AES. In Hans Dobbertin, Vincent Rijmen, and Aleksandra Sowa, editors, *AES Conference*, volume 3373 of *Lecture Notes in Computer Science*, pages 27–41. Springer, 2004.
11. Jovan Dj. Golic and Christophe Tymen. Multiplicative masking and power analysis of AES. In Kaliski Jr. et al. [12], pages 198–212.
12. Burton S. Kaliski Jr., Çetin Kaya Koç, and Christof Paar, editors. *Cryptographic Hardware and Embedded Systems - CHES 2002, 4th International Workshop, Redwood Shores, CA, USA, August 13-15, 2002, Revised Papers*, volume 2523 of *Lecture Notes in Computer Science*. Springer, 2003.
13. Paul C. Kocher. Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems. In Neal Koblitz, editor, *CRYPTO*, volume 1109 of *Lecture Notes in Computer Science*, pages 104–113. Springer, 1996.
14. Paul C. Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In Wiener [21], pages 388–397.
15. François Koeune and Jean-Jacques Quisquater and. A timing attack against Rijndael. Technical Report CG-1999/1, Université Catholique de Louvain, 1999.
16. Thomas S. Messerges. Securing the AES finalists against power analysis attacks. In Bruce Schneier, editor, *FSE*, volume 1978 of *Lecture Notes in Computer Science*, pages 150–164. Springer, 2000.
17. Gilles Piret and Jean-Jacques Quisquater. A differential fault attack technique against SPN structures, with application to the AES and KHAZAD. In Colin D. Walter, Çetin Kaya Koç, and Christof Paar, editors, *CHES*, volume 2779 of *Lecture Notes in Computer Science*, pages 77–88. Springer, 2003.
18. Kai Schramm, Gregor Leander, Patrick Felke, and Christof Paar. A collision-attack on AES: Combining side channel- and differential-attack. In Marc Joye and Jean-Jacques Quisquater, editors, *CHES*, volume 3156 of *Lecture Notes in Computer Science*, pages 163–175. Springer, 2004.
19. Kai Schramm, Thomas J. Wollinger, and Christof Paar. A new class of collision attacks and its application to DES. In Thomas Johansson, editor, *FSE*, volume 2887 of *Lecture Notes in Computer Science*, pages 206–222. Springer, 2003.
20. Sergei P. Skorobogatov and Ross J. Anderson. Optical fault induction attacks. In Kaliski Jr. et al. [12], pages 2–12.
21. Michael J. Wiener, editor. *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*. Springer, 1999.